

T1

尝试刻画充要条件：

- s 和 t 同时包含字符串 w ;
- s 和 t 不包含任何一个长度为 $|w| + 1$ 的最长公共子串。

我们希望根据是否包含 w 、包含 $|w| + 1$ 的子串类型，将所有的长度为 n 和长度为 m 的串划分为若干类。只需要同时包含 w ，且包含 $|w| + 1$ 的串交集为空即可。

那么尝试 DP。记 $dp_{i,s_1,S,0/1}$ 为：考虑了前 i 个字符，最后三个字符为 s_1 ，出现的长度为 $|w| + 1$ 的字符的集合为 S ， w 是否已经出现的方案数。

显然 S 只有 $2^{2^{|w|+1}}$ 种可能，每个状态也只有 $O(1)$ 个后继，直接转移可以接受。在 DP 之后，我们得到了 $f_{0,1,\dots,2^{2^{|w|+1}}-1}$ 和 $g_{0,1,\dots,2^{2^{|w|+1}}-1}$ ，我们求的就是 $\sum_{i \text{ and } j=0} f_i g_j$ 。

这个可以直接 and 卷积，或者直接高位前缀和即可。复杂度为： $O(Tn2^{2^{|w|+1}+|w|+1})$ 。

T2

弱化版是黑题，比较难绷。

考虑平凡的 DP，发现容易使用字典树或者哈希优化到 $O(m^2)$ 。实际上，如果你的常数足够优秀，可能可以多拿一些分，我尝试击杀了一些暴力，不过如果你获得了更多的分就假装这是《追忆 2》即可。

现在有两种转移，一个是在字典树上走后缀，一个是在字典树上走前缀。

第一步是进行根号分治：将串分为 $\leq B$ 和 $> B$ 两种。对于长度 $\leq B$ 的情况，我们可以直接暴力做到 $O(mB)$ 。 $> B$ 的情形，我们先考虑走后缀。

这时候考虑枚举 v ，对于每个 $> B$ 的串，找出从 v 往上走最远能走到哪。直接倍增显然是 $O(\frac{S}{B} \log n)$ 的，然而有更加聪明的做法：考虑对所有的 S_u 反串做后缀排序；直接使用倍增进行排序就是 $O((S + m) \log m)$ 的了（我比较懒，std 写的可能是 $\log^2$ ，不过这里并不重要）。利用前缀和优化，就可以完成转移。

然后考虑走前缀。我们希望对于每个点 u ，看往下走能走到哪个节点。

直接二分仍然可以做到 $O(\frac{S}{B} \log n)$ ，但是需要 umap 和哈希表之类的东西，常数会起飞。我们依旧介绍一个更加聪明的方法：继续考虑枚举 v 。枚举 T 之后，哪些 v 的祖先 u 可以走 T 并且停止在 v 呢？显然这样的 (u, v) 对是 $O(m)$ 的，只需要全部找到即可。也就是对于 v ，找到 T 的一段前缀，使得 T 的下一个字符并不在 v 的出边种，且 T 的这一段前缀是 S_v 的后缀。建出 AC 自动机后容易维护这个过程。

但是你要给 $u \rightarrow v$ 的链上加一个数，这个可以看做在 DFS 序数组上单点加区间查询，继续使用根号平衡即可做到 $O(m\sqrt{m})$ 。

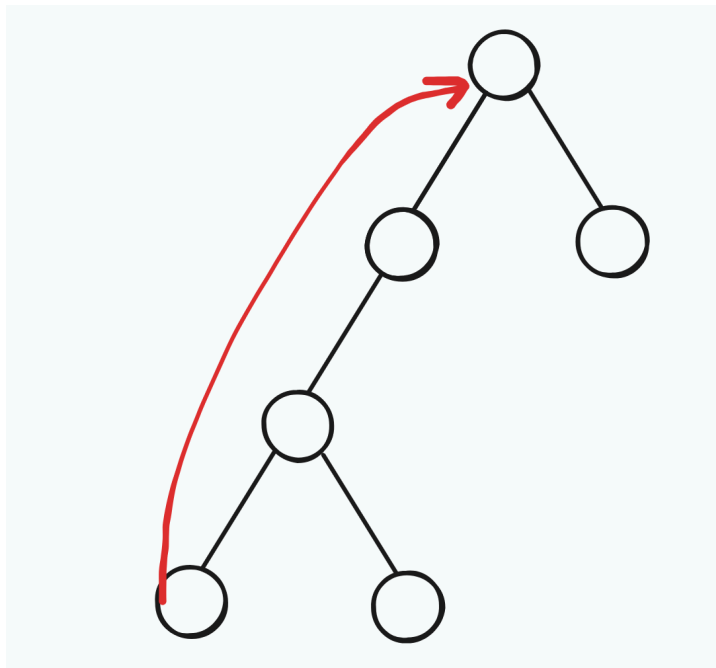
总复杂度也是单根号量级的。

由于各种原因，这道题时限相当充裕，你的 $O(m\sqrt{m} \log n)$ 如果卡过去了，那就恭喜你！/hec

T3

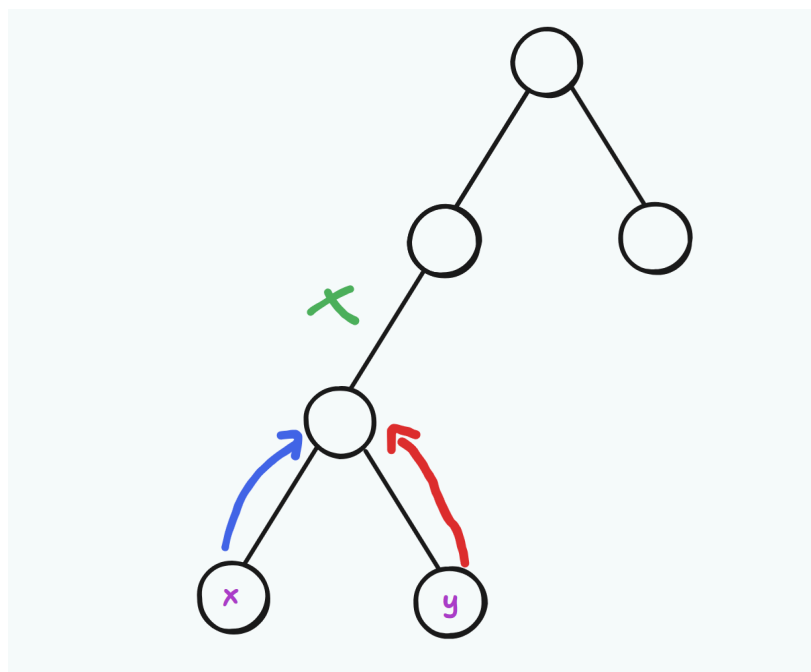
本题思维难度并不高，但是代码难度比较高。

此类问题很容易想到要建出虚树！不过在此之前，我们先要思考：一个点 u 什么时候会被 i 纳入势力范围内（只要 i 不是第一步就被灭了，容易证明只要不是第一轮被灭国，就不会被灭国）。



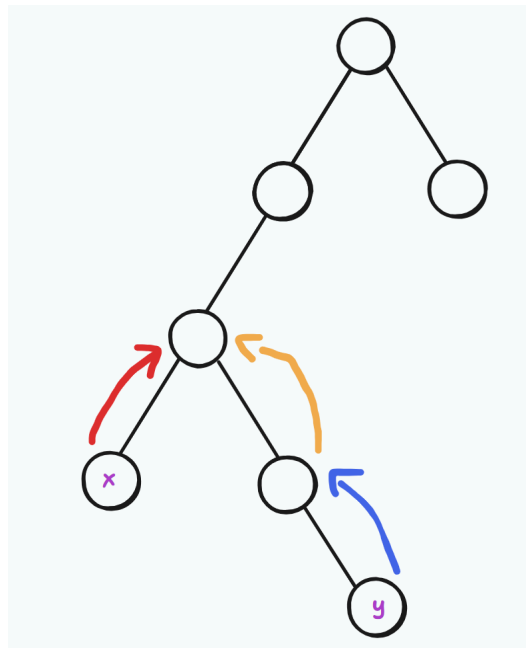
不妨把 u 放到根上，应该是某个 a_i 不断向上爬，打到了 u 。（当然，一开始要先处理灭国的情况）所以一个直观的想法是，找到离 u 最近的 a_i ，这样 a_i 应该是第一个打上来的！

然而这个结论是错的！原因是，可能会出现两个人在互相“争夺”，导致无法继续往上爬。



如上图所示， x 和 y （假设 x 在前）会不断抢夺 x 父亲的控制权，这样没有一个人能在父亲这里站稳脚跟，继续往上扩展。

所以，如果从下往上做的时候，当前离某个点最近的儿子不止一个（世子之争向来如此），就无法继续往上传递了。那么这就是唯一的情况了吗？



如果 y 的编号比 x 小，且 y 的深度恰好比 x 大一，虽然 x 先到达了 x 的父亲，但是在它继续扩展之前， y 会攻占它，防止 x 进一步扩展。所以，这种情况也需要判掉！手玩一下可以发现，这应该是全部情况。

回到“构建虚树”之后，我们先判断虚树中所有点的归属。发现这个其实是一个换根 DP 的过程。相当于记录二元组 (dis, id) ，求出所有儿子（或者说，后继，因为有换根的存在），维护不同子树二元组的最小值和次小值。这个东西反正具有交换律和结合律，因此可以通过前后缀和做到 $O(1)$ 的转移。算出 DP 数组之后，每个点就可以 $O(deg)$ 计算贡献了！

然后考虑不在虚树中的点。如果你做过《桂花树》这道题，你会知道：不在虚树中的点其实是有很多种情况的！一种是，卡在虚树的某条边上；一种是，挂在虚树的某条边上；一种是，挂在虚树的某个点上。这几个 case 都不怎么难，你会计算单点之后就应该知道他们怎么算了（但是有很多非常恶心的情况，比如边中间的一个点会有两种情况，也许会出现一个点挂出去的东西不能要了，这些都要继续倍增计算；以及，最后一个case 很容易漏，我最开始写的时候就漏掉了）。

复杂度应该为 $O((\sum k) \log n)$ ，但是常数比较大，所以时限也比较大。虽然 T3 的 std 比 T2 要短一些，但是细节多了不少。